

ECON1059 Seminar 6

Notes for the Seminar Series on Python Programming for Economics (ECON1059, Fall 2025)

Juncheng ZHANG, PhD Candidate in Economics, UNNC.

Research interest: Behavioral and Experimental Economics.

Email: Juncheng.ZHANG@nottingham.edu.cn

File Paths

Python needs a **file path** to locate files.

Two types of paths:

Type	Meaning
Relative	Relative to folder of current <code>.ipynb</code>
Absolute	Full system path (e.g., <code>/Users/.../Documents/...</code>)

Check current working directory:

```
import os
os.getcwd()
```

```
In [1]: import os
os.getcwd()
```

```
Out[1]: '/Users/stanley/Desktop/python FALL'
```

```
In [ ]: ├── python FALL/           ← This is my current working directory (cwd)
        │   ├── Seminar_6.ipynb   ← Running notebook
        │   │   ├── topic_6/      ← Subfolder
        │   │       └── simple_text_file.txt
```

```
└─ example_1.txt
└─ example_2.txt
```

Note: Use relative paths from the cwd

Reading Files

Use `open(path, "r")`.

- `read()` → read the entire file as one string
- `readlines()` → read all lines into a list
- **Loop line-by-line** → memory-efficient, recommended
- `with` → automatically closes the file when finished, recommended

```
In [2]: ## Example 1 – Read entire file

# My cwd is "python FALL/", if I need to open
# "example_1.txt" inside "topic_6/" subfolder,
# I use the relative path as below:

file_path = "topic_6/example_1.txt"

f = open(file_path, "r")
text = f.read()
print(text)
f.close()
```

Hi there!

Welcome to the seminar 6 :D

```
In [3]: ## Example 2 – Read all lines into a list

f = open(file_path, "r")
lines = f.readlines()
print(lines)
f.close()
```

```
['Hi there!\n', 'Welcome to the seminar 6 :D']
```

```
In [4]: ## Example 3 – Looping through a file (recommended)

f = open(file_path, "r")
for line in f:
```

```
    print(line)      # Each line ends with '\n'
f.close()
```

Hi there!

Welcome to the seminar 6 :D

In [5]: *## Example 4 – with automatically closes the file (recommended)*

```
with open(file_path, "r") as f:
    text = f.read()

print(text)
```

Hi there!

Welcome to the seminar 6 :D

`strip()` family removes characters from **string edges only**, not from the file itself. They return a **new string** → original unchanged.

Method	Removes from	Default removed chars	Example	Output
<code>strip()</code>	both sides	whitespace (<code>\n</code> , space, <code>\t</code>)	" hi ". <code>strip()</code>	"hi"
<code>lstrip()</code>	left side	whitespace	" hi ". <code>lstrip()</code>	"hi "
<code>rstrip()</code>	right side	whitespace	" hi ". <code>rstrip()</code>	" hi"

In [6]: *## Example 4 – Removing specific characters*

```
s = "hello!!!#@$"
print(s.rstrip("!#@"))  # Starts from the RIGHT end ('$'), last char is '$' → NOT in {"!", "#", "@"}

s = "hello!!!#@$!##@"
print(s.rstrip("!#@"))  # → "hello!!!#@$"

s = "hel@lo!!!#@$!##@"
print(s.rstrip("!#@"))  # → "hel@lo!!!#@$"

s = "@!@@$@hello!!!#@$"
print(s.lstrip("!#@"))  # → "$@hello!!!#@$"
```

hello!!!#@\$

hello!!!#@\$

hel@lo!!!#@\$

\$@hello!!!#@\$

Writing Files

Use `open(path, mode)`.

Modes:

Mode	Meaning
"w"	write → overwrite
"a"	append → add to end

Reading/Writing to file

1. Write some text to a file. Afterwards, read the text, add something new, and save it to the file. Finally, open your file using any text-editor and check its content. (Optional: try vary your answer by using either the "write" or the "append" mode when you "add something new".)
2. Create a .txt file with content of your choice using a word-processor. Save it in the same folder as your file in part 1. Write an algorithm that reads in the text in both files and writes both of them to a new file (in any order).
3. Suppose you have a .txt file with email addresses (one per line). You want to copy/paste all of them **in one go** to a particular application. Alas, this application doesn't let you directly copy/paste them! However, it turns out that it accepts a single line of text, where email addresses are separated by "; ", e.g.: "abc_1@x.com; abc_2@x.com; abc_3@x.com". Write a code that reads in your file and prints your addresses in the correct format to the screen for easy-copy pasting. You can create your own .txt file or use `PS6_email_addresses.txt` on moodle.

```
In [7]: ## Example 5 – Writing Files (Part of the ans to Q1.)
```

```
file_path = "topic_6/example_2.txt"

f = open(file_path, "w")    # create / overwrite
f.write("How you guys feeling today?\n")
f.close()
```

```
In [8]: ## Example 6 – Append to a file (Part of the ans to Q1.)
```

```
file_path = "topic_6/example_2.txt"

f = open(file_path, "a")    # open for append
f.write("We wish you enjoy our session!")    # append at end
f.close()
```

```
In [9]: ## Example 7 – Read the updated file (Part of the ans to Q1.)
```

```
file_path = "topic_6/example_2.txt"
```

```
f = open(file_path, "r")
text = f.read()
print(text)
f.close()
```

How you guys feeling today?
We wish you enjoy our session!

```
In [10]: ## Example 8 combined text files (Ans to Q2.)

combined_text = []

# Read the first file
f = open("topic_6/example_1.txt", "r")
combined_text += f.readlines()    # read all lines into list
f.close()

# Add a blank line between two files
combined_text.append("\n")

# Read the second file
f = open("topic_6/example_2.txt", "r")
combined_text += f.readlines()    # append all lines
f.close()

# Write all collected lines into a new output file
f = open("topic_6/example_1_2_combined.txt", "w")
f.writelines(combined_text)      # write list → file
f.close()

print(combined_text)
```

```
['Hi there!\n', 'Welcome to the seminar 6 :D', '\n', 'How you guys feeling today?\n', 'We wish you enjoy our session!']
```

`.join()`

`.join()` is a **string method** used to combine a list of strings into one string, using the caller string as a **separator**.

```
separator.join(list_of_strings)
```

```
In [11]: ## Example 9 – Using `.join()`
"; ".join(["apple", "banana", "orange"]) # → "apple; banana; orange"
```

```
Out[11]: 'apple; banana; orange'
```

```
In [12]: """.join(["H", "e", "l", "l", "o"]) # → "Hello"
```

```
Out[12]: 'Hello'
```

```
In [13]: # Ans to Q3.
f = open("topic_6/PS6_email_addresses.txt")

list_lines = f.readlines()
lines_clean = []
for line in list_lines:
    lines_clean.append(line.rstrip())

emails_as_text = "; ".join(lines_clean)
print(emails_as_text)
```

```
abc_1@nottingham.edu.cn; abc_2@nottingham.edu.cn; abc_3@nottingham.edu.cn
```

Dictionaries (dict)

- Dictionaries store data in **key-value pairs**(curly brackets). You can think of a real dictionary as a collection of **pairs** where each **key** maps to a **value**.
- Keys must be immutable (`str` , `int` , `float` , `bool` , `tuple` ; **NOT** `list` , `dict`)
- Values can be any data type
- Dictionaries are mutable, can add, update, or delete key–value pairs

In the real world:

From **Longman Dictionary of Contemporary English**

Related topics: *Newspapers, printing, publishing*

dic·tion·a·ry

/ˈdɪkʃənəri/ \$ -neri **noun** (plural **dictionaries**) [countable]

1 a book that gives a list of words in alphabetical order and explains their meanings in the same language, or another language

a German–English dictionary

In Python:

```
dict_1 = {key_1: value_1, key_2: value_2}
```

In [14]: `## Example 10 – Dictionary`

```
# Create
student = {
    "name": "Alice",
    "age": 19,
    101: "ID number"
}

# Accessing Values
print(student["name"])    # → Alice
print(student[101])      # → ID number

# Adding / Modifying
student["major"] = "Economics"    # add new pair
student["age"] = 20                # modify existing value
print(student)
```

Alice
ID number
{'name': 'Alice', 'age': 20, 101: 'ID number', 'major': 'Economics'}

In [15]: `# Deleting`
`del student["major"]`
`print(student)`

{'name': 'Alice', 'age': 20, 101: 'ID number'}

In [16]: `# Keys / Values / Items`
`print(student.keys())` `# dict_keys(['name', 'age', 101])`
`print(student.values())` `# dict_values(['Alice', 20, 'ID number'])`
`print(student.items())` `# dict_items([('name', 'Alice'), ('age', 20), (101, 'ID number')])`

dict_keys(['name', 'age', 101])
dict_values(['Alice', 20, 'ID number'])
dict_items([('name', 'Alice'), ('age', 20), (101, 'ID number')])

In [17]: `# Key existence`
`if "age" in student:`
 `print("Found age")`

Found age

- Create a dictionary of your choosing, access some of its values.
- Create a list of dictionaries that resembles a database for students. Each student should have (at least) a student_id, a name, and a major. Next, write a function that retrieves a particular student from the database based on their student_id.

- Part 1: skipped. See lecture notes for examples

```
In [18]: # Part 2.
# Create database of students
student_db = [
    {'student_id': 2023001, 'name': 'Jacky Shen', 'major': 'English Literature'},
    {'student_id': 2023002, 'name': 'James Ford', 'major': 'Ethics and Philosophy'},
]
```

```
In [20]: def get_student(student_id, db): #2023001 student_db
# for student in db:
#     if student['student_id'] == student_id:
#         return student # Note, this returns a dictionary!
```

```
In [21]: # Example of get_student()
student = get_student(2023002, student_db)
print(f"{student['name']} majors in {student['major']}")
```

James Ford majors in Ethics and Philosophy.

Hangman Game

- Create a txt file with a few words (one word per line) for our hangman game. Create a function that loads a random word.
- Optional: adjust your Hangman game so that it uses this function at the start the game

```
In [22]: import random

def load_secret_word(file_path):
    f = open(file_path) # default: `r`
    list_words = f.readlines()
    f.close()
    print(list_words) # add
    random_word = random.choice(list_words)
    print(random_word) # add
    random_word = random_word.rstrip('\n')
    return random_word
```

```
In [23]: # Test:
file_path = "topic_6/hangman_words.txt"
secret_word = load_secret_word(file_path)
print(f"Our randomly loaded word is: {secret_word}")
```

```
['olympics\n', 'espresso\n', 'mountains\n', 'grass\n', 'banana\n', 'skyscraper']  
olympics
```

Our randomly loaded word is: olympics

Alternatively you could use:

- `random_int = random.randrange(0, len(list_words))`, or
- `random_int = random.randint(0, len(list_words)-1)`
- And then: `random_word = list_words[random_int].rstrip('\n')`

`str.count(substring)`

Counts how many times `substring` occurs in the text.

`str.split(sep=None)`

Splits text into a list of words (defaults to splitting by whitespace).

```
In [24]: ## Example 11 – Counting Occurrences  
text = "they said they know"  
print(text.count("they"))    # → 2
```

2

```
In [25]: ## Example 12 – Splitting Text  
text = "they are fine."  
words = text.split()  
print(words)    # → ['they', 'are', 'fine.']
```

['they', 'are', 'fine.']

Counting Words in Text

- Without using the , Count the number of times the example text shared below uses "they". Afterwards, count the number of times it uses "nonsense". Do this with the build-in function `str.count(sub)` and by writing your own function for it.
- Hint 1: for your own function, split text into list using `str.split(sep=None)` .
- Hint 2: do you need to worry about capitalization of words?
- (very much optional: write this code without the split function. This will be quite tedious, but instructive. Note: I won't provide solutions for this)

Example text: "Mr. and Mrs. Dursley, of number four, Privet Drive, were proud to say that they were perfectly normal, thank you very much. They were the last people you'd expect to be involved in anything strange or mysterious, because they just didn't hold with such nonsense."

- Opening Lines to Chapter 1, The Boy Who Lived (Guess the book!)

```
In [26]: # Loading in Text
original_text = "Mr. and Mrs. Dursley, of number four, Privet Drive, were proud to say that they were perfectly normal, thank you very much. They were the last people you'd expect to be involved in anything strange or mysterious, because they just didn't hold with such nonsense."
original_text = original_text.lower() # Why is this important?
```

```
In [27]: # And using the in-built function
print(original_text.count("they"))
print(original_text.count("nonsense"))
```

3
1

- Let's start with a decent, albeit slightly insufficient solution
- Essentially, the code iterates over the list of words and check if the current element of the list matches the word we are looking for
- Please note: like most of you, I first wrote (a somewhat messy) code that didn't use a function. Once it worked, I turned it into a beautiful, general function. This question surely wasn't easy.

```
In [28]: def count_words(word_to_count, text):
count = 0

text_list = text.split()
# print(text_list) # Insightful! Uncomment to see what happens

for word in text_list:
    if word_to_count == word:
        count += 1

return count
```

```
In [29]: print(count_words("they", original_text))
```

3

```
In [30]: print(count_words("nonsense", original_text))
```

0

- It turns out that the last element of the list is "nonsense." not "nonsense"

- the `split()` function doesn't get rid of characters like `.,;!?;`
- → we need to clean up the original text a bit more
- The easiest way to do this is via `rstrip` where any elements you want deleted need to be put into a **single** text, e.g. `rstrip(".,;!?")`

```
In [31]: def count_words_improved(word_to_count, text):  
        count = 0  
  
        # Clean Text  
        text_list = text.split()  
        text_list_cleaned = []  
  
        for word in text_list:  
            text_list_cleaned.append(word.rstrip(".,;!?"))  
  
        # Search Text  
        for word in text_list_cleaned:  
            if word_to_count == word:  
                count += 1  
  
        return count
```

```
In [32]: print(count_words_improved("nonsense", original_text))
```

1